Computer Systems Literacy

Module 04

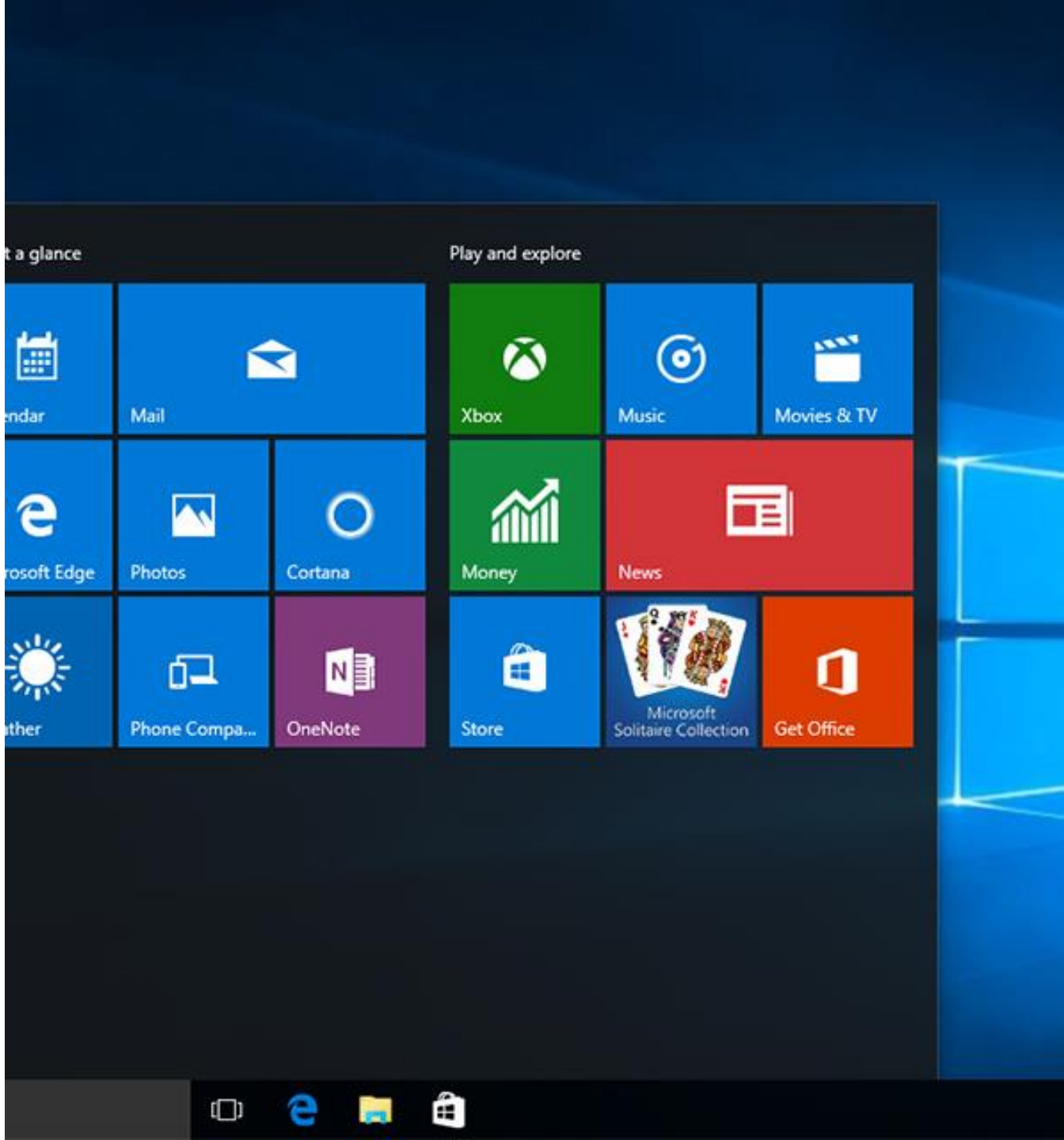
Unlocking the Terminal

Topics To Be Covered

- **The Shell Environment:** Understanding Bash/Zsh; the difference between a Terminal and a Shell.
- **The Global Path:** How the OS "finds" programs (The PATH variable).
- **Standard Streams:** Redirection (>), appending (>>), and the power of Pipes (|).
- **Permissions & Ownership:** The root user, sudo, and read/write/execute (rwx) security models.
- **Hands-on:** Building a "one-liner" script to filter, sort, and save system log data.

Graphical User Interface

Graphical User Interface (GUI) is a system of interactive visual components – such as icons, windows, menus, and pointers – that allows users to interact with electronic devices.



GUI

Graphical User Interface

Windows • Icons • Menus • Visual Interaction

Graphical User Interface

- **Purpose:** Provides a user-friendly, intuitive, and accessible way to navigate software without needing extensive technical or programming knowledge.
- **Functionality:** Enables multitasking by allowing users to open, view, and interact with multiple applications in different windows simultaneously.
- **Advantages & Disadvantages:** While GUIs are easier to learn and highly visual, they are more resource-intensive (requiring more memory and processing power) than text-based systems.

Command Line Interface

The command line interface (CLI), is a text-based method for interacting with your computer's operating system by typing commands.

CLI is not a relic of the past, but the primary tool for **automation**, **server management**, and developer efficiency.

```
[jim@palatinate:~]$ ls -l
total 36
drwxr-xr-x  2 jim jim 4096 Aug  3 19:42 bin
drwx------ 29 jim jim 4096 Oct 26  2021 chromium
drwxr-xr-x  3 jim jim 4096 Oct  8 09:16 Downloads
drwx------  7 jim jim 4096 Oct  7 13:35 Dropbox
drwxrwxr-x 12 jim jim 4096 Sep 29 20:47 hobby
lrwxrwxrwx  1 jim jim   21 Oct 25  2021 Log -> /home/jim/Dropbox/Log
drwxr-xr-x  9 jim jim 4096 Jul 29 09:43 Records
drwxr-xr-x  9 jim jim 4096 Aug  3 19:31 Resources
drwx------ 11 jim jim 4096 May 22 11:43 snap
drwxr-xr-x 11 jim jim 4096 Aug  7 15:57 Writing

[jim@palatinate:~]$ ls Writing/TheCodedMessage/
archetypes   content  public     run.sh  TECH_WRITING.md      themes     WRITIN
config.toml  layouts  resources  static  TECH_WRITING_OLD.md  upload.sh  WRITIN

[jim@palatinate:~]$ uname -a
Linux palatinate 5.14.0-1024-oem #26-Ubuntu SMP Thu Feb 17 14:35:50 UTC 2022 x8

[jim@palatinate:~]$ ping thecodedmessage.com
PING thecodedmessage.com (45.79.152.153) 56(84) bytes of data.
64 bytes from li1251-153.members.linode.com (45.79.152.153): icmp_seq=1 ttl=56
64 bytes from li1251-153.members.linode.com (45.79.152.153): icmp_seq=2 ttl=56
64 bytes from li1251-153.members.linode.com (45.79.152.153): icmp_seq=3 ttl=56
64 bytes from li1251-153.members.linode.com (45.79.152.153): icmp_seq=4 ttl=56
^C
--- thecodedmessage.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 12.234/13.721/15.797/1.372 ms

[jim@palatinate:~]$
```

Command Line Interface

- Shell/Terminal: The shell is the program that interprets the commands you type and tells the operating system what to do. Common examples include Bash (Linux, MacOS), Command Prompt (Windows) and PowerShell (Windows).
- Prompt: A sequence of characters (e.g., $, #, >) that indicates the shell is ready to accept a command.
- Command: A specific instruction given to the computer (e.g., ls or dir to list files)
- Scripting/Automation: Commands can be stored in a script file (like a batch file or a shell script) to automate repetitive or complex tasks, a major advantage over GUI.

Accessing the Command Line

Every major Operating System provides mechanisms to open the command line:

1. Windows
   - Press Windows key + R, type cmd or powershell, and press Enter.
   - Search for "Command Prompt" or "PowerShell" in the Start menu.
2. MacOS
   - Open your Applications folder, go to Utilities, and find Terminal.
   - Use spotlight search (Command + Space) and type Terminal.
3. Linux
   - Use the shortcut `Ctrl` + `Alt` + `T`.
   - Search for "Terminal" in your applications menu.

GUI vs CLI: The Paradigm Shift

The difference between choosing and constructing.

- The GUI is a restaurant menu – you can only order what is listed. The CLI is a kitchen – you can cook whatever you want if you have the ingredients.
- GUIs are intentionally limited; developers only include buttons for the most common tasks to keep the interface clean. The command line, however, exposes the full suite of "flags" and parameters for a tool.
- The CLI is incredibly "lean". It requires almost no RAM or CPU compared to a rendering engine for a modern "window" environment.

Read: Command Line Interface Explained - AWS

GUI vs CLI: The Paradigm Shift

Feature	Command Line (CLI)	Graphical User Interface (GUI)
Learning Curve	Steep; requires memorization.	Low; intuitive and visual.
Speed	Faster for complex/repetitive tasks.	Faster for simple, one-off tasks.
Resources	Minimal (Text-only).	High (Requires video rendering).
Flexibility	Highly customizable via scripting.	Limited to what the developer provides.

Why Use Command Line Interface

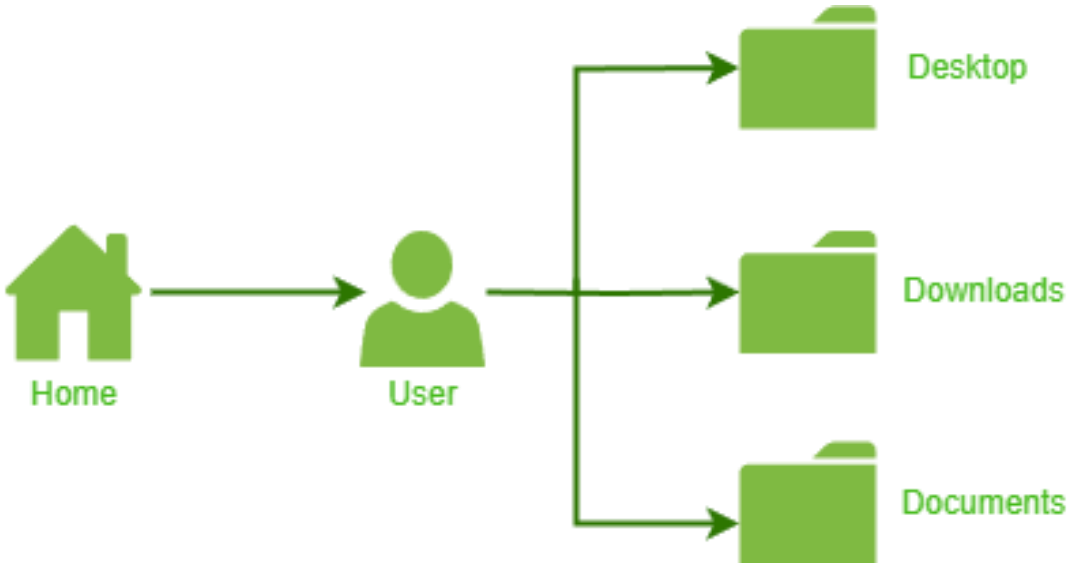
- **Efficiency and Speed:** Experienced users can perform tasks more quickly with a few keystrokes than with a GUI.
- **Automation:** Commands can be stored in scripts (shell scripts or batch files) to automate repetitive or complex tasks, saving time.
- **System Administration & Development**: CLIs are essential tools for system administrators and software developers for managing servers, configuring networks, and accessing system-level functions not available in a GUI.
- **Remote Access:** CLIs require fewer resources, making them ideal for managing remote systems over network connections, such as Secure Shell (SSH).
- **Piping and Redirection:** Users can direct the output of one command to become the input of another, enabling powerful data manipulation.

Navigation

Moving without seeing

Navigation in CLI is fundamentally about understanding and manipulating the file system hierarchy.

- Think in terms of **file systems**, **paths**, and **states**.
- Windows uses a hierarchical file system structured as a tree. At the top level are drives, such as C:\ and D:\
- You can view the hierarchy in the command line using the tree command.



Types of Paths

Paths in Windows use backslashes (`\`) as separators. There are two types of paths we must understand:

- **Absolute path:** A complete path starting from the drive root.
  Example: `C:\Users\Ankit\Documents`
- **Relative path:** A path relative to the current working directory.
  Example (if current directory is `C:\Users\Ankit`): `Documents`

Every CLI session maintains state – specifically the current working directory (CWD). All relative commands are interpreted based on this directory. The command to display the current directory in Windows is cd

Core Navigation Commands

1. Listing contents of the current folder – dir
   - dir displays files and directories in the current directory.
   - Outputs: file name, directory names (marked <DIR>), file sizes, and last modified timestamps.

```
C:>dir
 Directory of C:

2/18/2026  3:03:26 PM    <DIR>          .
2/18/2026  3:03:26 PM    <DIR>          ..
2/18/2026  3:03:10 PM    <DIR>       Program Files
2/18/2026  3:03:10 PM    <DIR>       Program Files (x86)
2/18/2026  3:03:10 PM    <DIR>       Users
2/18/2026  3:03:10 PM    <DIR>       Windows
2/18/2026  3:03:10 PM    76          readme.txt

     1 File(s)    76 bytes
     6 Dir(s)
```

Core Navigation Commands

1. Changing directories: cd
   - Moving into a subdirectory: cd <directory-name>
   - Move back one directory: cd .. (The .. symbol means parent directory)
   - Move back two directories: cd ..\..\
   - Move to root: cd \
   - Move to absolute path: cd C:\Users\Ankit\Desktop
   - To switch drives, go to root directory and run D:
2. Special navigation symbols:
   - . - current directory
   - .. - parent directory

Core Navigation Commands

1. Tab completion feature (high productivity)
   - Pressing tab autocompletes directory and file names.
   - This prevents typos and increases speed.
   - Example: type cd Doc and press tab. Auto-completes to cd Documents
2. Address paths with spaces. Windows directory often contains spaces (e.g., Program Files). You must use quotes for the command to work.
   - Example: cd "Program Files"
3. CLI navigation is deterministic and context-driven. Every command executes in a specific directory context. This is critical when we are running executables, compiling code, executing scripts, redirecting output, and managing logs.
   - Example: if you run python app.py - the command executes in the current directory. If you are in the wrong directory, it fails.

Navigation Exercises

1. Exercise 1: Navigate to your Desktop and list all files.
2. Exercise 2: Move into a folder on Desktop, then move back to parent directory.
3. Exercise 3: Switch to another drive (like D:) and list its root contents.
4. Exercise 4: Navigate to `C:\Program Files` (handle spaces correctly).

Navigation

Windows CLI navigation is not about memorizing commands. It is about understanding:

1. File system hierarchy
2. Path resolution
3. Session state
4. Context-dependent execution

Once you grasp these four ideas, the CLI becomes logical instead of intimidating.

File Management & Wildcards

Direct file system manipulation without abstraction.

In Windows CLI, you can execute commands that directly modify the NTFS file system. There is no explorer layer. When you delete something, it does not go to the Recycle Bin. It is immediately and permanently removed from the directory structure.

At a systems level:

1. A file name is an entry in a directory table.
2. When deleted, that entry is removed.
3. Data may remain on disk until overwritten, but recovery is unreliable.
4. Therefore, *Precision > Speed*

Command: del

File Management: Creating Directories

1. Directories are containers for files and other directories.
2. Creating new directories does not affect other files.
3. You can create nested directories.

Command: mkdir Project

Command: mkdir parent\child\grandchild

If intermediate directories don't exist, cmd creates it automatically.

Absolute path: mkdir C:\Temp\Test

Relative path: mkdir ..\Archive

File Management: Creating Files

1. To create an empty file: type nul > notes.txt
   - nul is a special device that outputs nothing.
   - The symbol '>' redirects the output into a file.
   - If the file exists, it overwrites it.
2. Alternative command: echo
   - Example: echo [your file content here] > filename.txt
   - This command uses echo to print the specified text and the redirect operator saves that output to the specified file.
   - If the file already exists, this will overwrite its contents.
3. Create/edit a file using notepad: notepad filename.txt
   - This command opens the Windows Notepad application.

> creates/overwrites.

>> appends.

```
copy con filename.txt
```

Type the command, then enter your text.

File Management: Copying Files

- Command: copy notes.txt backup.txt
- Copies file contents.
- Prompts before overwrite (unless suppressed)
- Can copy multiple files.
- Command: copy *.txt backup_folder\

File Management: Copying Folders

To copy folders or directories, use xcopy or robocopy (more advanced tools)

Example Command: xcopy folder1 folder2 /E /I

- /E copies subdirectories including empty ones.
- /I assumes destination is a directory

File Management: Move and Rename

Renaming Command: move old.txt new.txt

- Renaming and moving using the same command.
- Within same drive -> metadata update (fast)
- Across drives -> copy + delete (slower)

Move Command: move file.txt Archive\

Rename folder: move oldfolder newfolder

File Management: Deleting Files

Example Command: del file.txt

- No recycle bin. No undo.
- Immediate removal from directory listing.

Safer alternative del /P file.txt

- The /P prompts before deleting each file.

Delete multiple files of the same type: del *.txt

Delete directories: rmdir folder

Wildcards

Wildcards are expanded by cmd before execution.

1. Deletes all .jpg files in current directory: del *.jpg
2. Copies matching files only: copy report_2024_*.pdf Archive\
3. del file?.txt - Matches file1.txt, fileA.txt, but not file10.txt.

Critical Safety Pattern: Always preview before destructive wildcard use.

Example: dir *.jpg then del *.jpg

Remember: Preview -> Execute

Current Directory Context

- All commands operate relative to the current directory.
- If you are in C:\Users\Desktop
- del *.txt only deletes text files in Desktop.

Live Exercise

Create a folder named sandbox:

```
mkdir sandbox
cd sandbox
```

Create files:

```
type nul > file1.txt
type nul > file2.txt
type nul > image1.jpg
type nul > image2.jpg
```

List:

```
dir
```

Copy:

```
copy file1.txt backup.txt
```

Rename:

```
move file2.txt notes.txt
```

Preview wildcard:

```
dir *.jpg
```

Delete safely:

```
del /P *.jpg
```

Professional Safety Rules

1. Always confirm current directory before delete.
2. Use `dir` to preview wildcard matches.
3. Prefer `/P` until comfortable.
4. Avoid running destructive commands from root directories like C:
5. Create a test sandbox before experimenting.

Pipes and Streams

Think of the Windows Command Line (CLI) or PowerShell as a workshop. Streams are the conveyor belts carrying material, and Pipes are the connectors that join the different machines together. Let’s see how this digital plumbing works:

The 3 Standard Streams: Every time you run a command, Windows automatically opens three "channels" or streams for data to flow through.

Stream	Name	What it does	Default Source/Destination
stdin	Standard Input	Where the command gets its data.	Your keyboard
stdout	Standard Output	Where the command sends its successful results.	Your screen
stderr	Standard Error	Where the command sends error messages.	Your screen

1. Streams

Most beginners think commands "print to the screen". That's wrong. Commands write to **stdout** (standard output). The screen is just the default destination for **stdout**.

Similarly,

stdin -> standard input (default: keyboard)

stdout -> normal output (default: screen)

stderr -> error output (default: screen)

These are logical data streams, not visual concepts. (Add)

2. Pipes (|)

The Pipe symbol is the vertical bar on your keyboard. Its job is simple: It takes the stdout (output) of the command on the left and plugs it directly to the stdin (input) of the command on the right.

Without pipes, we would have to:

1. Run Command A
2. Save the result to a file.
3. Open Command B and tell it to read that file.

With a pipe, it happens instantly in memory: Command A | Command B

Example: tasklist | findstr "chrome"

2. Pipes (|)

Example : Filtering Output

1. dir | find "txt"
   - Dir produces output.
   - Find receives that output as input.
   - Find filters lines containing "txt"
2. type log.txt | find "Error"
   - type outputs file content (stdout)
   - find searches text.
   - The pipe connects them.

3. Redirection: Changing the Flow

Sometimes we do not want data to go to another command; we want it to go to a file (or come from one). This is called redirection.

1. Overwrite (>): Sends output to a file, deleting whatever was there before.
   - Example: `ipconfig > network_settings.txt`
   - Example: `dir > files.txt`
2. Append (>>): Sends output to a file but adds it to the end of the existing text.
   - Example: `dir >> files.txt`
3. Input (<): Tells a command to read data from a file instead of a keyboard.
   - Example: `sort < list.txt`

3. Redirection: Standard Error

By default, pipes (|) only pass **stdout**. If an error occurs, that message usually skips the pipe entirely and jumps straight to your screen.

Sometimes, we actually want to capture the errors (e.g., writing to a log file and want to see what went wrong). We can do this by a technique called stream redirection.

In CMD, use `2>&1`. This literally means "Send stream 2 (stderr) into the same place as stream 1 (stdout)."

Example: `mycommand.exe 2>&1 | findstr "Error"`

Why this Matters?

- This modularity is the secret sauce of power users.
- Instead of needing one giant program that does everything, you can string together five small, simple tools to perform a complex task.
- It's efficient, fast, and saves us from a lot of manual copy-pasting.

PowerShell

PowerShell is much more powerful because it does not "parse" text; we just ask for the property we want (.Name or .Size). This is the "magic" of PowerShell. In the old world (CMD or Linux Bash), every command outputs a long string of text. To get a specific piece of data – like a file size – you have to tell the computer: *"Look at line 5, skip 20 characters, and grab the next 10 digits."* That is **parsing**, and it's a headache because if the text format changes even slightly, your script breaks.

PowerShell skips the text and passes objects instead.

PowerShell vs CMD

If you want to find files larger than 1 GB:

1. We would have to use a complex for loop to scan every line of text, find the "size" column, and convert that text into a number to compare it. It's 10+ lines of messy code.
2. The PowerShell way: Get-ChildItem | Where-Object Length -gt 1GB
   - .Length: You aren't searching for text. You are asking the file object for its length property directly.
   - .Name: You aren't cutting characters out of a string; you are just asking for the "Name" property.

Fun Exercises (PowerShell)

1. The "Clean Up My Drive" Pipeline

This finds the 5 largest files in a folder so you can see what's eating your storage.

```
Get-ChildItem | Sort-Object Length -Descending | Select-Object -First 5
```

- `Get-ChildItem`: Grabs everything in the folder.
- `|`: Hands those items to the sorter.
- `Sort-Object Length -Descending`: Organizes them by size, biggest first.
- `|`: Hands the sorted list to the picker.
- `Select-Object -First 5`: Only shows you the top 5 results.

2. The "What's Lagging My PC?" Pipeline

If your computer feels slow, this shows you the top 10 processes using the most memory (RAM).

```
-Process | Sort-Object WorkingSet -Descending | Select-Object -First 10
```

Pro Tip: In PowerShell, `WorkingSet` is the technical term for the memory a program is actively using.

4. The "Quick Search" Pipeline

Need to find a specific line of text inside a massive log file?

```
Get-Content C:\Logs\System.log | Select-String "Error"
```

- `Get-Content`: Streams the entire file line by line.
- `Select-String "Error"`: Acts like a high-speed filter, only letting lines containing the word "Error" pass through to your screen.

PowerShell vs. CMD

Feature	Parsing (Text-based)	Objects (PowerShell)
Logic	"Find the 5th word in the 3rd line."	"Give me the `.Size`."
Fragility	Breaks if the layout changes.	Stays working because the object structure is constant.
Effort	High (lots of string manipulation).	Low (just call the property name).

Permissions & Superusers

An operating system enforces access control to protect system integrity, user data, and process isolation. Every file and directory has metadata defining **ownership** (who owns it), **group association,** and **permission bits** (what actions are allowed). This model ensures that not every user can modify system-critical resources.

User Identity and Privilege Levels

1. Users: Each process runs under a specific user identity (UID). That identity determines what files and system resources it can access.
2. Root (Superuser): In Unix-like systems, root is the superuser account. It has UID 0 and is not subject to normal permission restrictions.

Root and Sudo

In the context of Linux or Unix-like operating systems, the management of files is governed by a **Hierarchical Access Control** model. This system ensures system stability and security by adhering to the **Principle of Least Privilege**, where users are granted the minimum level of access necessary to perform their tasks.

1. Administrative Hierarchy (Root and Sudo): At the apex of the system's security architecture is the Superuser, traditionally known as root.
   - The Root Account: This is the system's administrative identity, assigned the User ID (UID) of 0. Root possesses "God-mode" privileges, meaning it can bypass all standard permission checks. It can read, modify, or delete any file on the system, regardless of ownership.
   - The sudo command: Standing for "superuser do", this utility facilitates temporary privilege escalation. It allows a standard user – provided they are in the authorized "sudoers" list – to execute a specific command with root-level authority.

The Permission Model (rwx)

Permissions are defined by three distinct actions that can be performed on a file or directory. Every file and directory has a permission structure consisting of three sets of permissions: User (owner), Group (a set of users), and Others (everyone else).

1. Each set contains three possible permission bits:
   a. Read (r)
   b. Write (w)
   c. Execute (x)
2. These bits are displayed using ls –l command (ls is the linux alternative to dir).
   a. Example output: -rwxr-xr--
   b. First character: regular file (-), directory (d). Other letters indicate special file types.

The Permission Model

Next nine characters (rwxr-xr--) grouped in three:

1. Owner permissions: rwx
2. Group permissions (r-x)
3. Others' permissions (r--)

You cannot access a file inside a directory unless you have 'execute' permission on that directory, even if the file itself is readable.

Read (r)

Files: Ability to view file content.
Directories: Ability to list directory content.

Write(w)

Files: Ability to modify contents.

- Directories: Ability to create, delete, or rename files inside the directory.

Execute (x)

Files: Ability to execute the file as a program/script.
Directories: Ability to enter the directory.

Changing Permissions (chmod)

The chmod (change mode) command is the mechanism used to alter these access rights. It can be utilized via two primary methods.

Symbolic Notation: This method uses mathematical operators to add or subtract permissions.

- `chmod +x script.sh` : Adds the execute permission to the file for all ownership tiers.
- `chmod u+w file.txt` : Grants the owner (user) the permission to write to the file.

Octal Notation: For more precise control, permissions are often represented by numbers, where Read = 4, Write = 2, and Execute = 1.

- `chmod 755 script.sh` : Where 7 (4+2+1) is full access for owner, 5 (4 +1) is read and execute for the group, and 5 (4 +1) is read and execute for others.

Why Permissions Matter

Permissions are the gatekeepers of the Operating System. Without them, a single rogue script or accidental command could delete the entire filesystem.

By separating "Identity" (who you are) from "Capability" (what you can do), the system remains robust against both human errors and malicious intent.

When a program runs, it inherits the permissions of the user who launched it. It cannot access files beyond that user's privilege unless special mechanisms are involved.

Programming 101

Syntax is the formal grammar of a programming language. It defines the exact structure and arrangement of symbols that make a statement valid.

For example, in Python: print("Hello") is a valid structure because:

- print is a defined keyword/function.
- Parentheses are required.
- The string is enclosed into quotes.

Computers do not “interpret” language the way humans do. They do not guess intent. They parse code according to strict grammatical rules defined in the language specification.

Programming 101 – Syntax Errors

- A syntax error occurs when code violates the formal grammar rules of the language.
- These errors are detected before the program runs (in compiled languages) or at runtime before execution proceeds (in interpreted languages).
- Syntax error = structure is invalid
  Semantic error = structure is valid but meaning is wrong
  Runtime error = program crashes during execution
  Logical error = program runs but gives wrong result
- Syntax defines how code must be written.
- A wrong syntax for print could be:
  - print(Hello)
  - print("Hello)

Thank you!